

BASE Properties

Chapter 7 | PostgreSQL Complete Guide Difficulty: Intermediate | Estimated Reading Time: 20 minutes

Table of Contents

- [Learning Objectives](#)
 - [Theory: BASE Properties](#)
 - [ACID vs. BASE Comparison](#)
 - [Eventual Consistency Deep Dive](#)
 - [BASE in Practice: Real Systems](#)
 - [Conflict Resolution Strategies](#)
 - [ASCII Visual Diagrams](#)
 - [Practical Examples](#)
 - [PostgreSQL Commands](#)
 - [SQL Examples](#)
 - [Common Mistakes](#)
 - [Best Practices](#)
 - [Performance Considerations](#)
 - [Interview Questions](#)
 - [Interview Answers](#)
 - [Hands-on Exercises](#)
 - [Solutions](#)
 - [Advanced Notes](#)
 - [Cross-References](#)
-

Learning Objectives

By the end of this chapter, you will be able to:

- Define BASE and explain each of its three properties
 - Contrast BASE with ACID and articulate the trade-offs
 - Explain eventual consistency and describe scenarios where it is acceptable
 - Describe common conflict resolution strategies in eventually consistent systems
 - Identify which real-world systems use BASE semantics and why
-

Theory: BASE Properties

BASE is an acronym that describes the consistency model used by many NoSQL and distributed databases. It was coined by Eric Brewer (the same person behind the CAP theorem) as a contrast to ACID:

- **B**asically **A**vailable
- **S**oft **S**tate
- **E**ventually **C**onsistent

BASE systems prioritize **availability and performance** over strict consistency. They sacrifice the guarantee that all nodes always have the same data in favor of always being able to serve requests.

The Origins of BASE

As internet-scale companies (Amazon, Google, Facebook) needed to handle billions of users across global data centers, traditional ACID databases couldn't scale horizontally with acceptable latency. The solution was to relax consistency guarantees — accept that different users might temporarily see different data, as long as the system eventually converges.

ACID vs. BASE Comparison

Property	ACID	BASE
Consistency	Strong, immediate	Eventual, relaxed
Availability	May sacrifice for consistency	Always available
Scalability	Vertical scaling primarily	Horizontal scaling
Transactions	Multi-row, multi-table ACID	Single-item operations
Complexity	Simpler for developers	Requires conflict handling
Use Case	Banking, ERP, e-commerce orders	Social media, DNS, CDN, caching
Examples	PostgreSQL, Oracle, MySQL	Cassandra, DynamoDB, CouchDB
Data Loss	Never (with durability)	Possible (window of inconsistency)
Latency	Higher (synchronization cost)	Lower (no coordination)

The Spectrum

ACID and BASE are not binary opposites. Many systems offer **tunable consistency** — you choose the consistency level per operation:

STRONG CONSISTENCY		WEAK CONSISTENCY	
<----->			
Linearizable (Spanner)	Serializable (PostgreSQL SERIALIZABLE)	Read-Your- Writes (DynamoDB default)	Eventual (Cassandra ONE)

Basically Available

The system guarantees availability. Every request receives a response, even if some nodes are down. The response might be:

- Slightly stale data (from a lagging replica)
- A partial result (not all shards available)
- Data from a local cache

The key guarantee: **the system does not refuse requests**. It degrades gracefully.

Example: DNS

DNS (Domain Name System) is a classic eventually consistent, basically available system:

- When you update a DNS record (e.g., point domain to new IP), the change propagates through global servers over minutes to hours
 - During propagation, different users get the old or new IP address
 - But DNS always responds — it never says "I can't answer this right now"
-

Soft State

The state of the system may change over time **without any input from the application**. Nodes converge toward consistency through background processes.

- Data may be in flux between nodes
- You cannot assume the state is stable at any given moment
- Background reconciliation processes continuously synchronize nodes

Contrast with ACID

In ACID systems, the database state is "hard" — once committed, data is stable and consistent. In BASE systems, data flows toward consistency but may be in transition.

Eventual Consistency

*Given that no new updates are made to a data item, all replicas will **eventually** converge to the same value.*

"Eventually" typically means milliseconds to seconds in practice. It does NOT mean the system might be inconsistent for days or years — the convergence time is bounded by network propagation and reconciliation intervals.

Types of Eventual Consistency

1. **Read Your Writes:** You always see your own writes (even if others don't yet)
 2. **Monotonic Read:** Once you see a value, subsequent reads won't show older values
 3. **Monotonic Write:** Writes from one session are ordered and applied in sequence
 4. **Write Follows Read:** A write that follows a read is applied to a state at least as recent as what was read
 5. **Causal Consistency:** Causally related operations are seen in order by all nodes
-

Conflict Resolution Strategies

When two nodes accept conflicting writes during a partition, the system must decide what to do when they reconnect:

1. Last Write Wins (LWW)

The write with the latest timestamp wins. Simple but can lose data.

```
Node A receives: {name: "Alice", ts: 10:00:01}
Node B receives: {name: "Alicia", ts: 10:00:02}
After merge: "Alicia" wins (later timestamp)
Problem: Clock skew can cause wrong results
```

2. First Write Wins

Whichever write was first (by timestamp or version) is kept. Also simple.

3. Merge (CRDT)

Conflict-Free Replicated Data Types (CRDTs) are data structures designed to merge automatically:

- **G-Set:** Grow-only set; merge = union
- **G-Counter:** Grow-only counter; merge = max per node, total = sum
- **LWW-Register:** Last-write-wins register
- **OR-Set:** Observe-Remove set (handles add+remove conflicts)

4. Application-Level Merge

The application receives both conflicting versions and decides. This is what Amazon's Dynamo paper described for shopping carts.

5. Version Vectors

Track which writes have been seen by using vector clocks:

```
Node A: {A: 2, B: 1} -- saw 2 writes from A, 1 from B
Node B: {A: 1, B: 3} -- saw 1 write from A, 3 from B
Conflict detected: divergent versions
```

ASCII Visual Diagrams

Diagram 1: ACID vs. BASE Behavior

```
ACID TRANSACTION
=====
Client --> DB Node A
      |
      | LOCK, VALIDATE, WRITE, COMMIT
      |
      | (If replica needed: WAIT for replica ACK)
      |
Client <-- SUCCESS (fully consistent)

Latency: Higher due to coordination
Consistency: Always strong

BASE OPERATION
=====
Client --> Node A (local region)
      |
      | WRITE (no waiting for other nodes)
      |
Client <-- SUCCESS (fast!)

      .....later.....

Node A --background replication--> Node B
```

Node A --background replication--> Node C

Latency: Low (no coordination)

Consistency: Eventual (seconds to minutes)

Diagram 2: Eventual Consistency Timeline

t=0 User updates profile photo

Node A (US-East): new_photo ✓

Node B (EU-West): old_photo (not yet propagated)

Node C (AP-East): old_photo (not yet propagated)

t=2s (replication in progress)

Node A: new_photo ✓

Node B: new_photo ✓ (replicated from A)

Node C: old_photo (network delay to Asia)

t=5s (fully converged)

Node A: new_photo ✓

Node B: new_photo ✓

Node C: new_photo ✓ (all nodes converged!)

Window of inconsistency: ~5 seconds

For a profile photo, this is ACCEPTABLE.

For a bank balance, this is NOT acceptable.

Diagram 3: Conflict in BASE System

PARTITION EVENT

=====

Network partition between Node A and Node B

Before partition:

Node A: inventory = 3

Node B: inventory = 3

During partition:

Client 1 --> Node A: Buy 2 items

Node A: inventory = 1 (local update)

Client 2 --> Node B: Buy 3 items

Node B: inventory = 0 (local update, OVERSOLD by 2!)

Partition heals:

Node A: inventory = 1

Node B: inventory = 0

CONFLICT! Which is correct?

RESOLUTION STRATEGIES:

LWW: Take the later timestamp (risky for inventory)

Manual: Trigger compensation (send refund)
CAS: Accept 0, trigger backorder for the extra 2 items

This is why AP (BASE) is wrong for inventory management!
Use CP (ACID) for inventory.

Practical Examples

Example 1: Social Media "Likes"

- User clicks "Like" on a post
- Count is updated on local node immediately
- Other nodes update asynchronously
- For a brief moment, different users see different like counts
- **Acceptable:** Users don't care if the count is 1,423 vs 1,425 for a few seconds

Example 2: Shopping Cart (Amazon Dynamo)

- User adds items on mobile, desktop, and tablet simultaneously
- Each device writes to its nearest node
- On checkout, all cart versions are merged (union of all items)
- No item is ever lost
- "Duplicate" items are de-duped at merge time

Example 3: Leaderboard

- Game scores are written to local node
 - Leaderboard is refreshed periodically from all nodes
 - There's a brief window where the leaderboard is slightly stale
 - **Acceptable:** Players understand leaderboards have a short delay
-

PostgreSQL Commands

```
-- PostgreSQL with asynchronous replication (BASE-like behavior)
-- On replica: reads may return slightly stale data
SELECT pg_is_in_recovery(); -- TRUE if this is a replica

-- Check current replay lag
SELECT
    now() - pg_last_xact_replay_timestamp() AS replication_lag,
    pg_last_wal_receive_lsn(),
    pg_last_wal_replay_lsn();

-- On primary: check if replicas have caught up
SELECT
    application_name,
    replay_lag,
    sync_state
FROM pg_stat_replication;

-- Configure hot standby to allow reads from replica
```

```
-- In postgresql.conf:
-- hot_standby = on
-- hot_standby_feedback = on
```

SQL Examples

Example 1: Eventually Consistent Counter Pattern

```
-- Distributed counter using per-node increments (G-Counter CRDT pattern)
CREATE TABLE distributed_counter (
    node_id    VARCHAR(50) PRIMARY KEY,
    count_val  BIGINT DEFAULT 0,
    updated_at TIMESTAMP DEFAULT NOW()
);

-- Each node only increments its own row
UPDATE distributed_counter
SET count_val  = count_val + 1,
    updated_at = NOW()
WHERE node_id = 'node-us-east';

-- Global total (read from any replica, may be slightly stale)
SELECT SUM(count_val) AS total_count
FROM distributed_counter;
```

Example 2: Idempotent Event Processing

```
-- Process events exactly-once using event_id deduplication
CREATE TABLE processed_events (
    event_id    UUID PRIMARY KEY,
    event_type  VARCHAR(100),
    payload     JSONB,
    processed_at TIMESTAMP DEFAULT NOW()
);

-- Idempotent: safely retry on failure
INSERT INTO processed_events (event_id, event_type, payload)
VALUES (gen_random_uuid(), 'user.login', '{"user_id": 123}')
ON CONFLICT (event_id) DO NOTHING;
```

Example 3: Read-Your-Writes Pattern

```
-- After a write, route reads to primary for the current session
-- This simulates "read-your-writes" consistency

-- Write to primary
SET LOCAL synchronous_commit = 'on';
UPDATE user_profiles SET avatar_url = 'new_url' WHERE user_id = 1;
```

```
-- Note the LSN after the write
SELECT pg_current_wal_lsn() AS write_lsn;

-- When reading from replica, wait until it has caught up to this LSN
-- pg_wal_lsn_diff(pg_last_wal_replay_lsn(), 'target_lsn'::pg_lsn) >= 0
```

Example 4: Soft-Deletes (Eventually Consistent Pattern)

```
-- Don't delete data immediately; mark as deleted
-- Allows replication to propagate before physical deletion
CREATE TABLE items (
  id          SERIAL PRIMARY KEY,
  content     TEXT,
  deleted_at  TIMESTAMP -- NULL means not deleted
);

-- "Delete" is just a soft mark
UPDATE items SET deleted_at = NOW() WHERE id = 42;

-- Queries filter soft-deleted rows
SELECT * FROM items WHERE deleted_at IS NULL;

-- Physical deletion happens much later (background job)
DELETE FROM items
WHERE deleted_at < NOW() - INTERVAL '7 days';
```

Example 5: Conflict Detection with Version Numbers

```
-- Optimistic concurrency for BASE-style systems
CREATE TABLE profiles (
  user_id  INT PRIMARY KEY,
  name     VARCHAR(100),
  version  BIGINT DEFAULT 1
);

-- Read with version
SELECT name, version FROM profiles WHERE user_id = 1;
-- Returns: "Alice", version=5

-- Update only if version matches
UPDATE profiles
SET name = 'Alice Updated', version = version + 1
WHERE user_id = 1 AND version = 5;

-- Check if update succeeded (0 rows = conflict, retry)
GET DIAGNOSTICS updated_rows = ROW_COUNT;
```

Example 6: Append-Only Log (BASE-Friendly Pattern)


```

-- Append-only log: never update, only insert
-- Consistent by design (each node can independently append)
CREATE TABLE audit_log (
    log_id      BIGSERIAL PRIMARY KEY,
    occurred_at TIMESTAMPTZ DEFAULT NOW(),
    entity_type VARCHAR(50),
    entity_id   INT,
    action      VARCHAR(50),
    old_data    JSONB,
    new_data    JSONB,
    node_id     VARCHAR(50) DEFAULT current_setting('app.node_id', TRUE)
);

-- Every change is an insert, never an update
INSERT INTO audit_log (entity_type, entity_id, action, old_data, new_data)
VALUES ('order', 1001, 'status_change',
        '{"status": "pending"}',
        '{"status": "shipped"}');

```

Example 7: Last Write Wins with Timestamp

```

-- LWW merge: keep the most recently updated version
CREATE TABLE user_settings (
    user_id     INT PRIMARY KEY,
    settings    JSONB,
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- LWW upsert: only update if the incoming timestamp is newer
INSERT INTO user_settings (user_id, settings, updated_at)
VALUES (1, '{"theme": "dark"}', '2024-01-15 10:30:00')
ON CONFLICT (user_id) DO UPDATE
SET settings    = EXCLUDED.settings,
    updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > user_settings.updated_at;

```

Example 8: Eventually Consistent Read via Read Replica

```

-- Application pattern: reads to replica, writes to primary
-- In connection string or application routing:

-- Write connection (primary):
-- postgresql://user:pass@primary-host:5432/db

-- Read connection (replica, may be up to N seconds behind):
-- postgresql://user:pass@replica-host:5432/db?options=-c%20hot_standby%3Don

-- Check staleness on replica:
SELECT

```

```

CASE
    WHEN (now() - pg_last_xact_replay_timestamp()) < INTERVAL '5 seconds'
    THEN 'Fresh'
    ELSE 'Stale: ' || EXTRACT(SECONDS FROM now() -
pg_last_xact_replay_timestamp()) || 's lag'
END AS replica_freshness;

```

Example 9: Merge Two Versions (Application-Level)

```

-- When two versions of a shopping cart merge
CREATE TABLE cart_items (
    cart_id    UUID,
    product_id INT,
    quantity   INT,
    added_at   TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (cart_id, product_id)
);

-- Merge cart from device 2 into cart from device 1
-- Strategy: take the MAX quantity (user likely wanted more, not less)
INSERT INTO cart_items (cart_id, product_id, quantity)
SELECT 'cart-device1', product_id, quantity
FROM cart_items
WHERE cart_id = 'cart-device2'
ON CONFLICT (cart_id, product_id) DO UPDATE
SET quantity = GREATEST(EXCLUDED.quantity, cart_items.quantity);

```

Example 10: Monitoring Eventual Consistency Window

```

-- Track when changes were written vs when they became visible on replica
CREATE TABLE replication_monitor (
    check_id    SERIAL PRIMARY KEY,
    written_at   TIMESTAMPTZ DEFAULT NOW(),
    visible_at   TIMESTAMPTZ,
    latency_ms   INT GENERATED ALWAYS AS (
        EXTRACT(MILLISECONDS FROM (visible_at - written_at))::INT
    ) STORED
);

-- Insert from primary
INSERT INTO replication_monitor (written_at) VALUES (NOW());

-- On replica, update when visible (separate monitoring job)
UPDATE replication_monitor
SET visible_at = NOW()
WHERE visible_at IS NULL
AND check_id = (SELECT MAX(check_id) FROM replication_monitor);

```

Common Mistakes

Mistake 1: Using BASE for Financial Data

Money transfers, inventory deductions, and ticket reservations require ACID. Using an eventually consistent system for these operations leads to overselling, double-spending, and inconsistent balances.

Mistake 2: Ignoring the Conflict Resolution Requirement

BASE systems don't resolve conflicts automatically (unless using CRDTs). Every BASE system that accepts concurrent writes needs a defined conflict resolution strategy.

Mistake 3: Treating "Eventually" as "Probably"

Eventual consistency guarantees convergence — it's not a probabilistic "might converge". Given no new writes and an operational network, all nodes WILL converge to the same value.

Mistake 4: Confusing BASE with "No Transactions"

BASE systems typically support single-item ACID operations. The relaxation is in cross-node, cross-item consistency, not within a single data item.

Mistake 5: Not Testing Partition Scenarios

Developers often test BASE systems in normal (no-partition) conditions where everything appears consistent. The divergence only appears during and after partitions.

Mistake 6: Using LWW Without Clock Synchronization

Last-Write-Wins requires accurate timestamps. Unsynchronized clocks (even NTP drift) can cause newer writes to be overwritten by older ones.

Best Practices

1. **Choose BASE only when eventual consistency is acceptable:** Analyze your data's consistency requirements before choosing a BASE system.
 2. **Use CRDTs for counters, sets, and flags:** Design data structures that merge automatically without conflicts.
 3. **Always implement idempotency:** Operations in BASE systems may be retried. Make them safe to execute multiple times.
 4. **Use version vectors or timestamps for conflict detection:** Know when conflicts exist before trying to resolve them.
 5. **Define your consistency window SLA:** "Eventually" should be bounded. Specify: "All nodes converge within 5 seconds under normal conditions."
 6. **Monitor replication lag:** Alert when the eventual consistency window exceeds your SLA.
 7. **For PostgreSQL with async replicas:** Route read-critical queries to the primary; use replicas only for read-tolerant workloads.
-

Performance Considerations

- BASE systems eliminate coordination overhead → dramatically lower write latency
 - Write throughput scales linearly with nodes (no coordination bottleneck)
 - Read latency is low when reading from the nearest node
 - Conflict resolution has computational cost — simple strategies (LWW) are faster than complex ones (vector clocks)
 - CRDT merge operations can be expensive for large data structures
-

Interview Questions

1. What does BASE stand for?
 2. How does BASE differ from ACID?
 3. What is eventual consistency and when is it acceptable?
 4. What is a CRDT and why is it useful in BASE systems?
 5. What is the trade-off between ACID and BASE in terms of availability and performance?
 6. Name three real-world systems that use BASE semantics.
 7. What is Last Write Wins (LWW) and what are its risks?
 8. Why is idempotency important in BASE systems?
-

Interview Answers

Q1: BASE = Basically Available (system always responds), Soft State (data may be in flux as nodes converge), Eventually Consistent (all nodes will agree on values given no new updates).

Q4: CRDTs

Conflict-Free Replicated Data Types are data structures mathematically designed to merge automatically without conflicts. They require all merge operations to be commutative, associative, and idempotent. Examples: G-Counter (increment-only), OR-Set (add/remove set), LWW-Register, MV-Register. Used by Redis, Riak, and distributed collaboration tools like Google Docs.

Q5: ACID vs. BASE Trade-off

ACID sacrifices availability and performance for consistency: writes are slower (must coordinate with replicas), the system may block during failures, and horizontal scaling is complex. BASE sacrifices consistency for availability and performance: writes are fast (no coordination), the system stays up during partitions, and it scales horizontally. The choice depends on whether your business can tolerate temporary inconsistency.

Hands-on Exercises

Exercise 1: CRDT Counter

Implement a G-Counter in PostgreSQL that works across multiple nodes. Each node has its own row; total is the sum.

Exercise 2: LWW Register

Implement a Last-Write-Wins key-value store in PostgreSQL that updates only when the incoming timestamp is newer.

Exercise 3: Idempotent Event Processor

Create a table of events. Write an idempotent processor that inserts events exactly-once, even when called multiple times with the same event.

Exercise 4: Replication Lag Monitor

Write a query that alerts when async replica lag exceeds 10 seconds.

Exercise 5: Cart Merge

Simulate two shopping cart versions from different devices. Write a merge query that takes the maximum quantity per product.

Solutions

Solution 1

```
CREATE TABLE g_counter (  
  node_id  VARCHAR(50) PRIMARY KEY,  
  val      BIGINT DEFAULT 0  
);  
INSERT INTO g_counter VALUES ('node1', 0), ('node2', 0), ('node3', 0);  
  
-- Node 1 increments its counter  
UPDATE g_counter SET val = val + 1 WHERE node_id = 'node1';  
  
-- Total count (BASE: sum across all nodes)  
SELECT SUM(val) AS total FROM g_counter;
```

Solution 2

```
CREATE TABLE lww_store (  
  key       VARCHAR(200) PRIMARY KEY,  
  value     TEXT,  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
CREATE OR REPLACE FUNCTION lww_set(p_key TEXT, p_value TEXT, p_ts TIMESTAMPTZ)  
RETURNS VOID AS $$  
BEGIN  
  INSERT INTO lww_store (key, value, updated_at) VALUES (p_key, p_value, p_ts)  
  ON CONFLICT (key) DO UPDATE  
  SET value      = EXCLUDED.value,  
    updated_at   = EXCLUDED.updated_at  
  WHERE EXCLUDED.updated_at > lww_store.updated_at;  
END;  
$$ LANGUAGE plpgsql;  
  
SELECT lww_set('user:1:theme', 'dark', NOW());
```

```
SELECT lww_set('user:1:theme', 'light', NOW() - INTERVAL '1 hour'); -- Ignored
(older)
SELECT * FROM lww_store WHERE key = 'user:1:theme'; -- Still 'dark'
```

Advanced Notes

The BASE Spectrum in PostgreSQL

PostgreSQL itself is ACID, but you can implement BASE-like patterns:

- **Async replication** → Basically Available, Soft State, Eventually Consistent at the replica level
- **Background jobs** → Soft state (queued operations)
- **Logical replication** → Multi-master with conflict resolution

When to Use BASE vs. ACID in the Same System

Many production systems use both:

- **ACID (PostgreSQL):** Orders, payments, inventory (mission-critical consistency)
- **BASE (Redis/Cassandra):** User sessions, recommendation data, activity feeds, analytics counters

This is **polyglot persistence** — use the right tool for each job.

Cross-References

- **Previous:** [06 - ACID Properties](#)
 - **Next:** [08 - PostgreSQL Introduction](#)
 - **Related:** [05 - CAP Theorem](#) — Theoretical foundation for BASE
 - **Related:** [02 - Types of Databases](#) — NoSQL systems that use BASE
 - **See Also:** [09 Administration/04 replication.md](#) — Async replication in PostgreSQL
-